NSBM Green University

CS107.3 Object Oriented Programming with C#

Ms. Lashika Chamini

15/08/2025

A WINDOWS FORMS BASED STUDENT MANAGEMT SYSTEM

GROUP I

CONTENTS

Contents

Problem Statement 1

Requirements 4

Design 6

Personal Reflection 8

Future Work 11

Snap Shots of the Work 15

YouTube Video Link 22

A WINDOW FORM BASED STUDENT MANAGENT SYSTEM.

Problem Statement

In today's educational landscape, managing data is essential for smooth academic and administrative operations. However, many institutions, especially those with limited technology, still use outdated methods like paper filing systems, handwritten records, or unconnected spreadsheets to handle student information. These methods may work on a small scale, but they have significant drawbacks: they **are time-consuming, prone to errors, hard to maintain, and at risk of data loss or damage.**

As student populations grow, managing individual records—like personal information, enrollment details, academic performance, and attendance—becomes more complex. Manual systems make it hard to quickly access specific information, ensure data consistency, and keep data secure. Additionally, many lack proper tools for tracking changes or generating reports, which slows down decision-making.

Without a centralized, secure, and **user-friendly digital platform**, daily operations become inefficient. Teachers and administrative staff waste time searching for, updating, and verifying information which could be automated. Also, manual data entry systems often lead to incorrect or incomplete data being stored due to a lack of error handling.

To tackle these challenges, a Student Record Management System (SRMS) is needed. This system would centralize student information, allow for fast and accurate data retrieval, enable secure updates, and connect easily to a database backend. It should be designed for scalability, allowing for future feature expansions like attendance tracking or integration with learning management systems. The solution should also follow Object-Oriented Programming (OOP) principles to guarantee modularity, maintainability, and ease of future development.

A WINDOW FORM BASED STUDENT MANAGENT SYSTEM.

INTRODUCTION

The Student Record Management System (SRMS) is a Windows Forms application built with C# and the .NET Framework. It connects to a Microsoft SQL Server database using ADO.NET for effective data handling. The main aim of the project is to give educational institutions a dependable, efficient, and secure way to manage all parts of student information. By combining an easy-to-use interface with a structured backend database, the SRMS removes the problems of manual record-keeping and improves overall data management.

In traditional educational settings, administrators often struggle with large amounts of student data housed in various registers, files, or spreadsheets. These systems can become inefficient and lead to data duplication, inconsistency, and loss. The SRMS solves these issues by providing centralized storage, which keeps all student information in a single secure database that can be accessed at any time. This **centralization cuts down on redundancy, boosts accuracy, and allows for quicker decision-making.**

From a technical standpoint, the **SRMS** showcases key software development concepts. The system uses OOP principles like:

Encapsulation: Grouping related properties and methods into specific classes to control data access.

Abstraction: Hiding complex database operations behind simpler, reusable methods.

Inheritance: Reusing common code structures for different entities and expanding functionalities as needed.

Polymorphism: Allowing flexible and maintainable code through method overriding and overloading.

Moreover, the system integrates exception handling to effectively manage unexpected situations like invalid user inputs, database connection failures, or missing records. This ensures stability and smooth user experience, even in challenging scenarios.

Functionally, the SRMS supports CRUD operations—**Create, Read, Update, and Delete**—allowing users to add new student records, access existing information, make changes, and delete outdated entries. The interface is user-friendly, making it easy for people with limited technical skills to use the system. The system can also be expanded to include additional features such as:

- Advanced search and filtering for large datasets.
- Report generation for academic or administrative needs.
- Role-based access control to safeguard sensitive data for authorized personnel.
- Integration with other systems like attendance trackers or learning portals.

From an institutional perspective, implementing a system like SRMS brings various benefits**: improved operational efficiency, greater accuracy of records, better use of staff time, and compliance with current data security rules. The system also enhances communication between administrative staff, teachers, and students by making sure that accurate, up-to-date information is always accessible.**

This project serves as a practical answer to real-world educational data management challenges. It also gives the development team valuable hands-on experience in professional software engineering practices. This strengthens skills in event-driven programming, database integration, and teamwork while delivering a product that meets industry standards and academic needs.

Requirements

1. Functional requirements

Student Record Management (CRUD Operations)

- Create: Users can add new student records, including details such as name, student ID, date of birth, address, contact number, and course.
- Read: Users can view all student records in a tabular format or search for specific entries.
- Update: Users can modify existing student details when information changes or errors are found.
- Delete: Users can remove outdated or incorrect student records from the system.

Database Integration

The system must be connected to a relational database (Microsoft SQL Server) for storing and retrieving all student data.

- Use ADO.NET components like SQL Connection, SQL Command, and SQL Data Adapter for communication with the database.
- Store all student details in a dedicated table with properly defined data types and constraints.
- Maintain data integrity through primary keys (e.g., Student ID).

User Interface (UI) Requirements

- Forms for adding, editing, viewing, and deleting student details.
- Clearly labelled fields and buttons (e.g., “Save,” “Update,” “Delete,” “Search”).
- Menu or navigation bar for switching between modules.

2. Non-Functional

These define system performance, security, and usability standards:

Performance Requirements

The system should load and display search results within 2 seconds for up to 1,000 records.

Database operations should run efficiently without noticeable delays.

Usability Requirements

The interface should be easy to use for non-technical users.

Use consistent button layouts, colors, and labels to make navigation simple.

Security Requirements

Implement role-based access if multiple user types are added in future versions.

Prevent SQL injections by using parameterized queries in all database operations.

Reliability and Availability

The system should handle unexpected failures smoothly without crashing.

Ensure data integrity during CRUD operations.

Scalability

The system should be designed to support additional features without major changes to its structure.

Maintainability

Use a modular code structure with OOP principles for easy maintenance and updates.

Design

The design of the Student Record Management System (SRMS) aims to create a structured, modular, and scalable software solution. It combines an easy-to-use interface with a dependable database backend. The design process takes a layered approach. This makes sure that each part of the system is organized and independent, which enhances maintainability and future scalability.

ARICHITECTURAL OVERVIEW

The SRMS is built on a three-layer design:

Presentation Layer (User Interface Layer)

This is the Windows Forms user interface from which users may interact with the system.

It contains forms to add, display, edit, and delete student records.

Controls such as text boxes, combo boxes, and data grids are used for input and display.

Buttons invoke calls to events that forward user requests to the business logic layer.

Business Logic Layer (Controller Layer)

This layer contains the necessary logic of the system.

It accepts user input, validates as per validation rules, and processes correct data before any database interaction.

It employs Object-Oriented Programming (OOP) concepts like encapsulation and abstraction to keep the code structured that is reusable.

Error handling is performed here to deal with wrong inputs or exceptions in the system.

Data Access Layer (Database Layer)

Manage all interactions with the Microsoft SQL Server database using ADO.NET.

It does CRUD operations—Create, Read, Update, Delete—via SQL queries.

Uses parameterized queries to prevent SQL injection and provide data safety.

All database connections are opened and closed for performance.

Personal Reflection

Working on the Student Record Management System project was one of the most valuable experiences in my journey as a student of Object-Oriented Programming. From the early planning stages to the final testing and presentation, I improved my technical skills and gained important insights into teamwork, problem-solving, and time management.

At the start of the project, I felt excited but also a bit nervous because the task seemed large and required integrating several concepts like C# Windows Forms, database connectivity, and OOP principles. Initially, I thought creating forms and connecting them to a database would be easy, but I quickly realized that it required careful planning, a clear understanding of design, and attention to detail.

One of my biggest lessons was how to approach project design in an organized way. Before coding, our group discussed the structure, features, and roles of each member. This helped me understand the importance of software design principles. Defining requirements early can save a lot of time later. I personally contributed to designing the database tables and setting up the Microsoft SQL Server connection. This improved my knowledge of SQL queries, table relationships, and the importance of using primary keys and constraints for data integrity.

During development, I learned how important it is to apply OOP concepts effectively. For example, using classes to represent student data, encapsulating attributes, and separating business logic from the database layer made our code more organized and easier to maintain. I also practiced exception handling to ensure the application managed invalid inputs or database errors gracefully, rather than crashing. This was something I had only read about before but never applied in a real project.

Working with ADO.NET for database connectivity was another highlight. I learned how to use SQL Connection, SQL Command, and SQL Data Adapter to interact with the database. More importantly, I learned to use parameterized queries to prevent SQL

injection attacks. This not only made the application more secure but also gave me a sense of writing professional, production-level code.

Teamwork was key to our project's success. We divided tasks based on our strengths but also supported each other when someone was stuck. I found that explaining a concept to a teammate not only helped them but also deepened my own understanding. Communication was essential. Regular updates and discussions kept us on track and helped us avoid last-minute issues.

However, the journey had its challenges. One of the first issues we faced was inconsistent database connections when moving the project between different computers. This taught me the importance of using relative paths and consistent connection strings. Another challenge was ensuring the UI design looked uniform across different forms. Initially, some forms seemed mismatched, but after receiving feedback, I paid extra attention to ensure the color schemes, fonts, and button styles were the same.

Looking back, I am proud of how far we came from the initial idea to the final working application. The experience taught me to break a big problem into smaller, manageable parts, document my work clearly, and test every feature thoroughly before calling it “done.”

If I were to improve the project in the future, I would want to add features like role-based login, attendance management, and report generation in PDF format. I also believe integrating the system with a cloud database could make it accessible from multiple devices, which would be helpful for real-world use in schools and universities.

In conclusion, this project was more than just an academic assignment; it was a practical lesson in software development, teamwork, and problem-solving. I leave this experience feeling more confident in my programming abilities and more prepared

for real-world projects. It has motivated me to keep learning and explore new technologies that can make systems like this even better.

Future Work

While the current version of the Student Record Management System (SRMS) meets its main goals of managing student information through a Windows Forms application with a database, there are several ways to improve its functionality, performance, and real-world use. These improvements can guide future work and upgrades to the system.

1. Role-Based Authentication and Access Control

Right now, the system does not distinguish between different types of users. A future version could have a role-based login system where administrators, teachers, and clerks have different access levels. For instance:

- Administrator: Full access to all records and system settings.

- Teacher: Access to student academic data but restricted from changing administrative details.

- Clerk: Limited to adding and updating student personal information.

This feature would boost security, data privacy, and accountability by allowing only authorized personnel to access sensitive information.

2. Attendance and Academic Performance Modules

The current SRMS mainly stores student personal and course details. In the future, we could develop additional modules to manage:

- Attendance tracking: Teachers can mark attendance daily, and the system can generate monthly or yearly reports.

- Grades and performance records: This would store exam results, GPA, and subject-wise marks for easy tracking of progress.

These additions would make the SRMS a more complete academic management system instead of just a personal record database.

3. Advanced Reporting and Data Visualization

Future versions of the SRMS might include a reporting engine that generates:

- Printable student lists and progress reports.
- Course enrollment summaries.
- Graphical visualizations like charts for attendance trends or grade distribution.

Connecting with libraries like Microsoft Report Viewer or third-party charting tools could improve the system's ability to display data clearly and professionally.

4. Cloud Database Integration

Currently, the system uses a local Microsoft SQL Server database, which restricts access to the computer where it is installed. Moving the database to a cloud solution such as Azure SQL Database or MySQL on AWS would allow multiple users to access it from different locations. This would be especially beneficial for institutions with multiple campuses or for staff working from home.

5. Mobile and Web-Based Access

To make the system more accessible, a future version could include:

- A mobile app for Android/iOS that allows authorized users to quickly check or update student records.
- A web portal that replicates the features of Windows Forms, enabling remote access via a browser.

This would require adjusting the backend to work with APIs and could also incorporate responsive design principles for better usability.

6. Backup and Restore Functionality

Data safety is crucial for any record management system. Adding an automatic backup feature that stores a copy of the database daily or weekly would help protect against accidental data loss. An easy restore function would ensure that data can be recovered quickly in case of a system failure.

7. Multi-Language Support

For use in areas with diverse languages, the SRMS could support multiple interface languages. This would make the system more inclusive for educational institutions serving different linguistic communities.

8. Enhanced Search and Filtering

While the current system allows basic searches by name or ID, future upgrades could add:

- Advanced filters for searching by course, year of enrollment, grade range, or attendance percentage.

- Real-time search suggestions as users' type.

This would make finding records in large datasets much faster and more user-friendly.

9. Integration with Learning Management Systems (LMS)

Many schools and universities use LMS platforms like Moodle or Google Classroom. The SRMS could be updated to synchronize with these platforms, automatically updating student enrollment data and grades between systems.

Conclusion of Future Work

The improvements outlined above would change the SRMS from a basic student data management tool into a fully featured educational management platform. These enhancements would not only make the system more useful and scalable but also keep it aligned with modern technological trends in education. By gradually incorporating these features, the system could adapt to the evolving needs of institutions while remaining relevant, efficient, and secure over the long term.

If you want, I can now also prepare a "Scope of the Project" section, so your report flows perfectly from Problem Statement to Introduction to Requirements, Design, Future Work, and Conclusion without any gaps.

Snap Shots of the Work

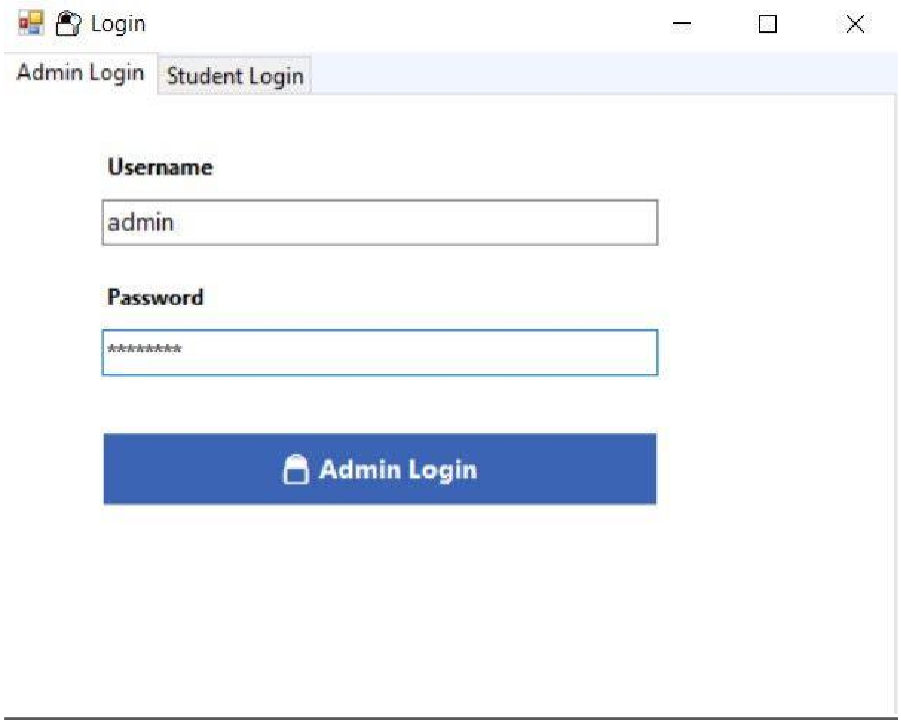

Figure 1 Admin login:

The default page of the app that appears as soon as it runs and displays a simple form to fill out details including username and password of the admin to login as an admin.

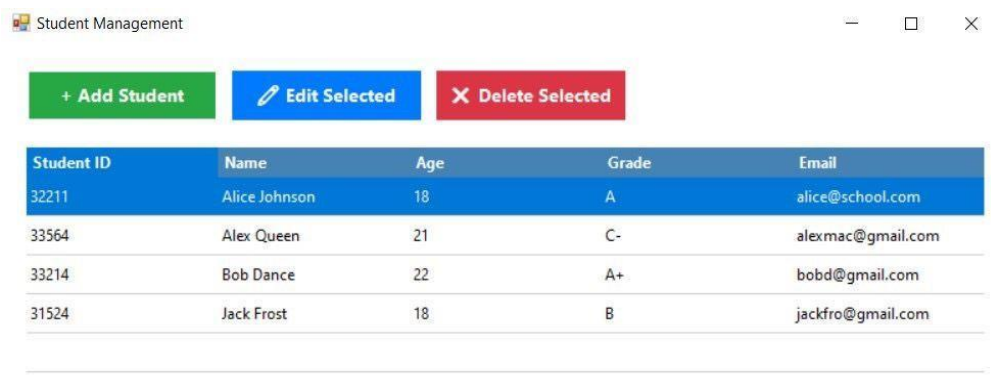

Figure 2 Admin Dashboard:

When the admin login with the correct credentials he/she is directed to the admin dashboard page displaying all the functions which they can perform and all the details of previously saved students.

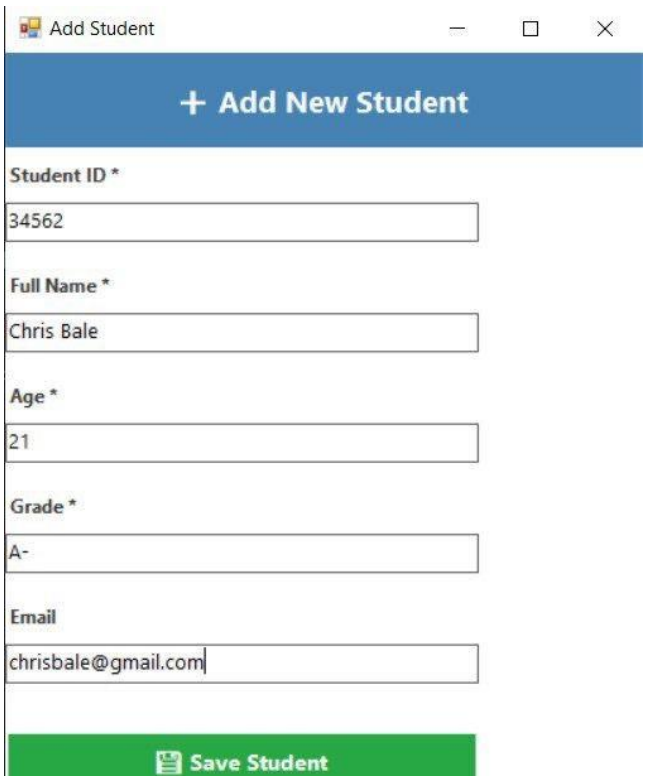

Figure 3 Add New Student:

By clicking on 'Add Student' on top left corner of the admin dashboard they are directed to this Add New Student Page. This provides the admin with a blank form to fill out all the relevant credentials of the student that should be added and then displaying it in the dashboard through a click to 'Save Student' .

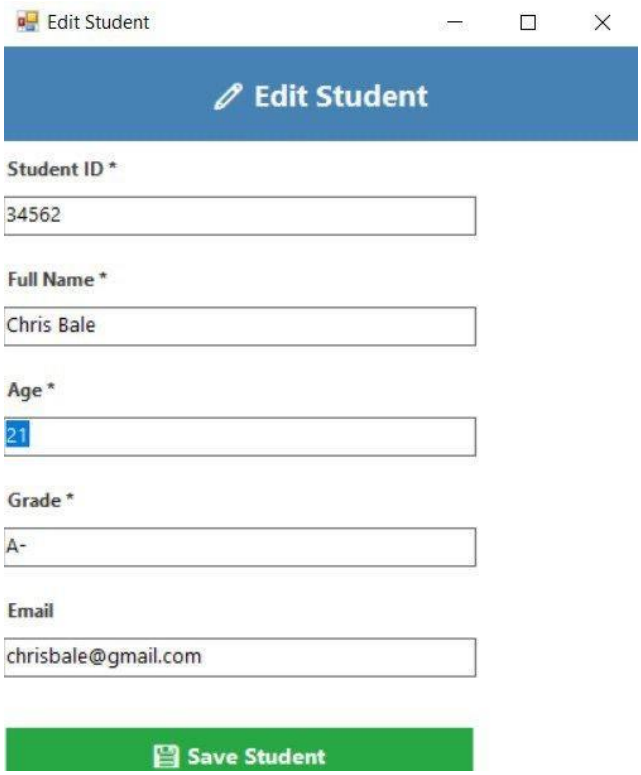

Figure 4 Edit Student:

First the admin must select a chosen row from the student detail table and then click on 'Edit Selected' . Then the admin is provided with a form which was already filled in based on the selected student. The admin can erase and enter new details which they need to change and click on 'Save Student' to view the updated details of the relevant student in the table.

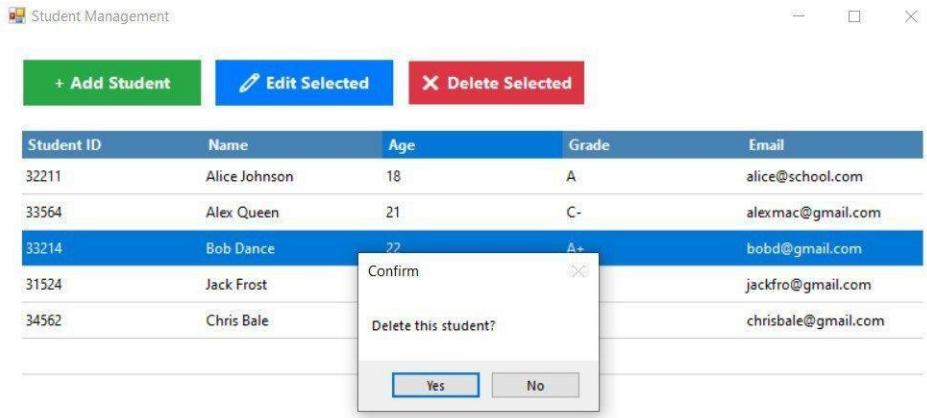

Figure 5 Delete Student:

Same as in editing a student, in deleting a student from the database the admin must select on the relevant row and then click on 'Delete Selected' on the top. Then it will send the user a verification box to confirm whether to delete the details of the selected student with a simple yes/no response request.

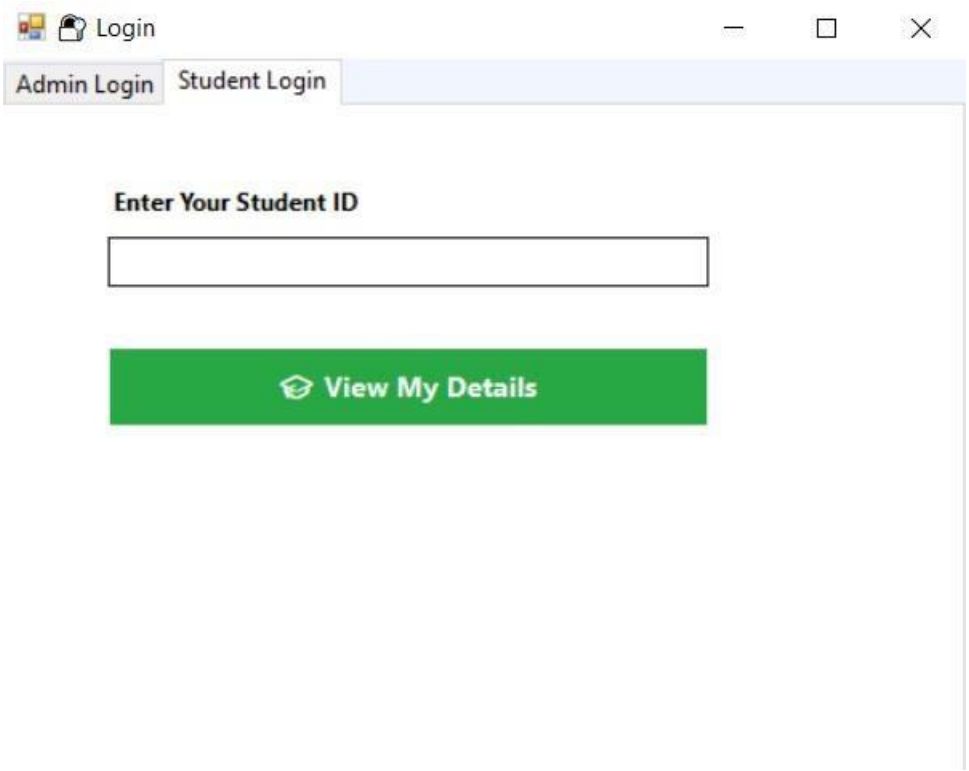

Figure 6 Student Login:

In the very first page where we meet the 'Admin Login', next to is the Student Login panel. Theres not much detail to enter there as a student, only their registered Student ID. After entering the correct student ID they simply need to click on 'View My Details' box.

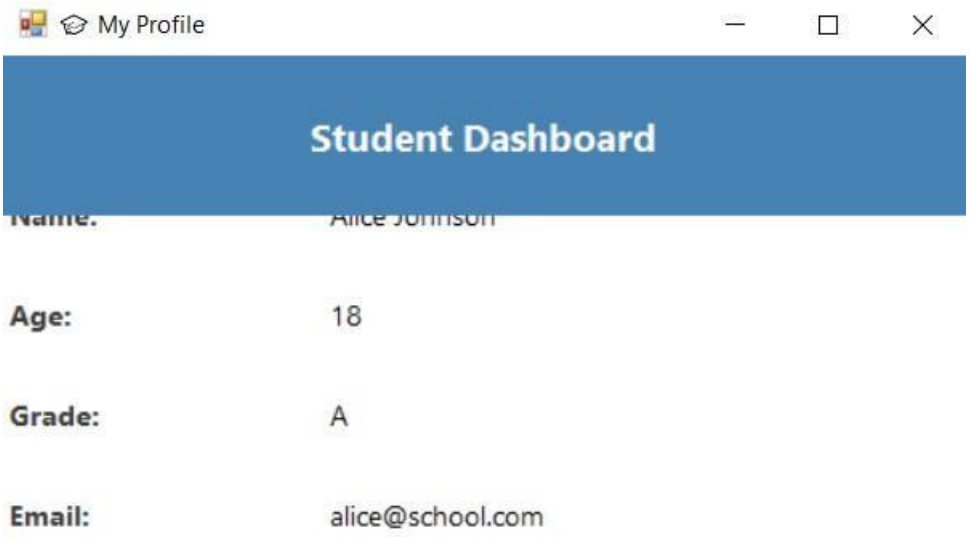

Figure 7 Student Dashboard:

If the student entered a valid and an existing student ID they will be displayed with their relevant details needed including their final grade he/she achieved in this Student Dashboard page.

YouTube Video Link

Link: https://youtu.be/RBdq-ikabjM